

Implementation and experimentation with Dynamic Staging

LP2, 2nd May 2026

CHING Long Tin, YEUNG Sin Chun

Motivation	1
Compile-Time vs. Run-Time Work	2
Multi-Stage Programming	4
Class specialization	7
Overview	9
Implementation	14
Benchmarking	39
Bibliography	41

Compile-Time vs. Run-Time Work

Programmers often write code in a clear, abstract style, even when parts of it could be computed during compilation.

Original program

```
1 fun dot(xs, ys) = mls
2   if xs is
3     Nil then 0
4     Cons(x, xt) and ys is
5       Cons(y, yt) then
6         x * y + dot(xt, yt)
7
8 fun dotWith3(v) =
9   dot([1, 0, 2], v)
```

Residual program (after specialising on [1, 0, 2])

```
1 fun dotWith3(v) = v(0) mls
  + 2 * v(2)
```

Compile-Time vs. Run-Time Work

The recursion, the pattern matches, the multiplication by 0 — all known at compile time. The residual program contains only the work that genuinely depends on the runtime input v .

Goal: let the programmer keep the abstract version, and have the compiler peel away the static layer automatically.

Multi-Stage Programming

Well-known optimization technique [1]

```
1 fun power(n, x) = if n is
2   0 then .<1> .
3   else .< .~x * .~(power(n - 1, x)) > .
4 fun power2 = .! .<(x => .~(power(2, .<x> .)))> .
```

In here, green annotates code to be executed in the next stage, and gray executed in the current stage.

Multi-Stage Programming

Rule for $.!$: evaluate until the whole code fragment is in the next stage, then move it to the current stage.


$$.!\left(\left(\left(x\right)\right)\right) = .!\left(x\right) = x$$

During evaluation, we are able to pre-compute certain parts of the code, reducing runtime.

Multi-Stage Programming

1 `x => power(2, x)`

mls

2 `x => if 2 is 0 then 1 else x * power(2 - 1, x)`

3 `x => x * power(1, x)`

4 `x => x * if 1 is 0 then 1 else x * power(1 - 1, x)`

5 `x => x * x * if 0 is 0 then 1 else x * power(0 - 1, x)`

6 `x => x * x * 1`

7 `fun power2 = x => x * x * 1`

Class specialization

```
1 class B(val x) with
2   fun f() = ...
3 // ...
4 if x is
5   1 then new D1(1)
6   2 then new D2(2)
7
8 x.f()
```

```
...           ...
8   if x is mls
9     D1 then x.D1_f1()
10    D2 then x.D2_f1()
11    D3 then x.D3_f1()
```

Traditional Multi-Stage Programming tracks types.

Even if x has a known class shape, we cannot remove matching arms.

Specialization is done through typing, so we know that $x : B$, but we do not know which specific derived class of B is used.

Class specialization

```
1 data class Foo(x, y) extends Bar(x+1, y+1)
2
3 if new Foo(1, 2) is
4   Bar(x, y) then ...
```

mls

Under single inheritance, matching arms runs into problems. [2]

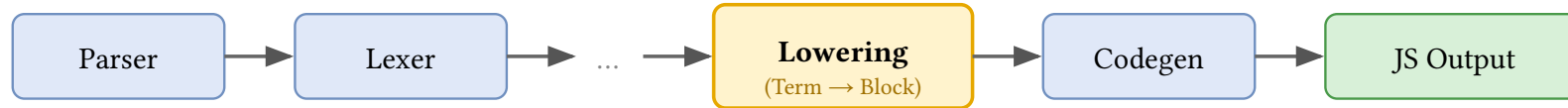
(Class splitting)

```
1 class Foo$1$2 extends Bar
2 class Bar$2$3
```

mls

Motivation	1
Overview	9
MLscript Compiler	10
Approach	12
Shape definition	13
Implementation	14
Benchmarking	39
Bibliography	41

MLscript Compiler

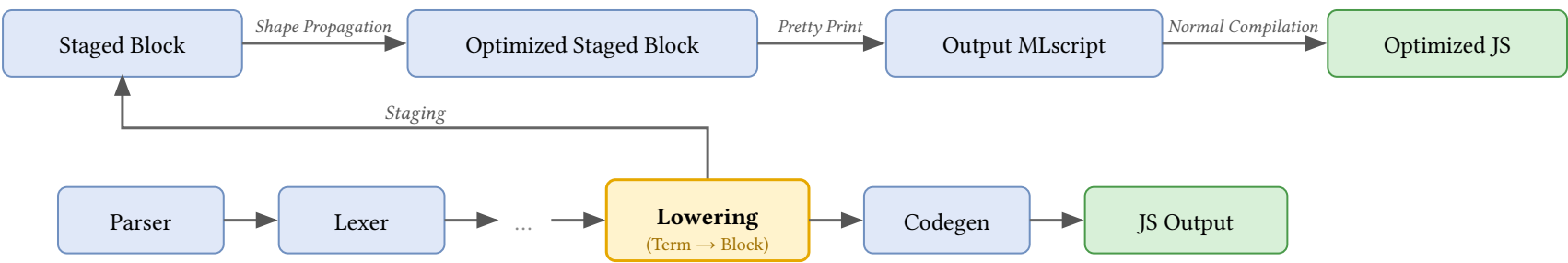


- Lexer/Parser: converts source code into AST
- Elaborator/Resolver: Deal with references
- **Lowering**: converts AST to a simplified format
- Codegen: Turn AST into executable code (JavaScript).

Lowering Pass

We implement staging a transformation from Scala Block $\Rightarrow$ Scala Block.

MLscript Compiler



||||| 05b79df

Approach

Two phases: Instrumentation / Shape Propagation

Shape definition

We focus on tracking the shape of the Paths and Results.

$$s ::= \iota \mid \mathbf{dyn} \mid [\overline{s}] \mid C(\overline{n : s}) \mid \perp \mid s \cup s$$

```
1 fun f(p, cond) =  
2   let a = 42  
3   let b = new C(p, 2)  
4   let c = if cond then [1, 2, 3] else C(1, 2)  
5   let d = if b is C then 0 else 1
```

mls

Motivation	1
Overview	9
Implementation	14
What's this for?	15
Staging	16
Instrumentation	21
Shape Propagation	22
Printing Staged Block	37
Benchmarking	39
Bibliography	41

What's this for?

Example 1

```
1 staged module Math with
2   fun pow(@dynamic x, n) = if n is
3     0 then 1
4     else x * pow(x, n-1)
5   fun sq(x) = pow(x, 2)
```

mls

...

...

```
5   fun sq(x) = x * x * 1
```

mls

This example touches on shape tracking, match arm removal, and nested specialization.

Staging

```
1 case class Return(res, implct) extends Block
2 case class Match(scrut, arms, dflt, rest) extends Block
3 case class Assign(lhs, rhs, rest) extends Block
```

 Scala

```
1 class Block with
2   constructor
3   Return(res, implct)
4   Match(scrut, arms, dflt, rest)
5   Assign(lhs, rhs, rest)
```

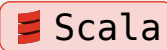


For any Scala Block data, we can recreate the same structure with **Staged Block**.

Staging

```
fun pow(x, n) = if n is 0 then 1 else x * pow(x, n-1)
```

```
1  val pow = Symbol("pow"); val x = Symbol("x"); val n = Symbol("n")
2  val t1 = Symbol("tmp"); val t2 = Symbol("tmp")
3  val sub = Symbol("-"); val mul = Symbol("*")
4  FunDefn(pow, [x, n], Scoped(HashSet(t1, t2), Match(Ref(n),
5    [[ Lit(0), Return(Lit(1)) ]],
6    Assign(t1, Call(sub, [n, Lit(1)]),
7    Assign(t2, Call(pow, [x, t1]),
8    Return(Call(mul, [x, t2]))
9    )
10  ), End())
11  ))
```



As long as we have the corresponding constructor, we can copy over the structure to the Staged Block.

Staging

```
fun pow(x, n) = if n is 0 then 1 else x * pow(x, n-1)
```

```
1  let pow = Symbol("pow"); let x = Symbol("x"); let n = Symbol("n")
2  let t1 = Symbol("tmp"); let t2 = Symbol("tmp")
3  let sub = Symbol("-"); let mul = Symbol("*")
4  FunDefn(pow, [x, n], Scoped([t1, t2], Match(Ref(n),
5    [[ Lit(0), Return(Lit(1)) ]],
6    Assign(t1, Call(sub, [n, Lit(1)]),
7    Assign(t2, Call(pow, [x, t1]),
8    Return(Call(mul, [x, t2]))
9    )
10  ), End())
11  ))
```

mls

As long as we have the corresponding constructor, we can copy over the structure to the Staged Block.

Staging

We perform structural induction to stage each type of Scala Block.

`Value.Lit(lit)`

1 `ValueLit(lit)`

mls

Example: $1 \mapsto \text{ValueLit}(1)$

`Symbol(name)`

1 `Symbol(name)`

mls

Example:

- $x \mapsto \text{Symbol}("x")$
- $C \mapsto \dots?$

This is not enough for staging symbols. We'll revisit this case later on.

Staging

We perform structural induction to stage each type of Scala Block.

`Value.Lit(lit)`

```
1 ValueLit(lit)
```

mls

Example: $1 \mapsto \text{ValueLit}(1)$

`Value.Ref(sym)`

```
1 let l = sym
```

mls

```
2 ValueRef(l)
```

Example: $x \mapsto \text{ValueRef}(\text{Symbol}("x"))$

`Select(qual, name)`

```
1 let q = qual
```

mls

```
2 let n = name
```

```
3 Select(q, n)
```

Example: $x.p \mapsto \text{Select}(\text{ValueRef}(\text{Symbol}("x")), \text{Symbol}("p"))$

Staging

```
Tuple([x0, x1, ...])
```

```
1 let s0 = x0
```

mls

```
2 let s1 = x1
```

```
...
```

```
...
```

```
4 Tuple([x0, x1, ...])
```

The other Scala Block cases are similar, staging the parameters of the Scala Block by induction and recreating the corresponding Staged Block counterpart.

Staging

Staging Symbols

- ▶ We need more information about the Symbol in the original stage

Add reference to current value in the symbol

C.x

```
1 let CSym = ClassSymbol("C", C)
2 Select(ValueRef(CSym), Symbol("x"))
```

mls

Add redirection within current stage to allow access for next stage

```
1 staged class A with
2   fun f() = M.f()
3   val redirect_M = M
```

mls

Staging

Staging Symbols

- Uniqueness of Symbols

We want Symbols for the same object in the previous stage to be unique in the current stage across functions and compilations.

For local symbols, we can maintain a map during staging to reuse a staged symbol.

x + x

```
1  let x = Symbol("x")
2  // let x1 = Symbol("x")
3  Call(ValueRef(Symbol("+")), [[ValueRef(x), ValueRef(x)]])
```

mls

Staging

Staging Symbols

- Uniqueness of Symbols

We want Symbols for the same object in the previous stage to be unique in the current stage across functions and compilations.

For class and module symbols, we need to cache and use first instance of a symbol within the staged code.

`M.f()`

```
1 let MSym = symbolMap.check(ModuleSymbol("M", M))  
2 Call(Select(ValueRef(MSym), Symbol("f")), [[]])
```

mls

Instrumentation

Insert some auxiliary helper variables/functions to the module for shape propagation.

```
1  staged module Math with
2    val funCache = new Map()
3    val generatorMap = new Map([["pow", pow_gen]])
4    fun pow_staged() = ...
5    fun pow_gen(x, n) =
6      specialize(funCache, "f", pow_staged, [x, n])
7    fun propagate() =
8      let dyn = ...
9      pow_gen(dyn, dyn)
10     sq_gen(dyn)
```

mls

Shape Propagation

A **Shape** captures what we statically know about a value at compile time.

$$s ::= \iota \mid \mathbf{dyn} \mid [\bar{s}] \mid C(\overline{n : s}) \mid \perp \mid s \cup s$$

We write $\llbracket s \rrbracket$ to denote the shape of an expression, distinguishing it from actual values.

```
1 fun f(p, cond) =
```

mls

```
2   let a = 42
```

$\llbracket 42 \rrbracket$

```
3   let b = C(p, 2)
```

$\llbracket C(\mathbf{dyn}, 2) \rrbracket$

```
4   let c = if cond then [1, 2] else C(1, 2)
```

$\llbracket [1, 2] \cup C(1, 2) \rrbracket$

```
5   let d = if b is C then 0 else 1
```

$\llbracket 0 \rrbracket$

Shape Propagation

Before the formal rules, let's give a simple trace of shape propagation.

```
1  class C(val n)
2  staged module Simple with
3    fun f(x) =
4      let y
5        if x is C then y = x.n
6        else y = 0
7      y + 1
8  fun test()      = f(C(2))
9  fun test2(dyn) = f(C(dyn))
10 fun test3()     = f(0)
```

mls

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let `y`: extend ctx with $y \mapsto \llbracket \perp \rrbracket$

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let `y`: extend ctx with $y \mapsto \llbracket \perp \rrbracket$
2. if `x` is `C`: `filter( $\llbracket C(n: 2) \rrbracket$ , C) =  $\llbracket C(n: 2) \rrbracket$` so then is viable; `rest =  $\llbracket \perp \rrbracket$` so else is dead

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

$y \mapsto \llbracket 2 \rrbracket$

1. let `y`: extend ctx with $y \mapsto \llbracket \perp \rrbracket$
2. if `x` is `C`: `filter( $\llbracket C(n: 2) \rrbracket$ , C) =  $\llbracket C(n: 2) \rrbracket$` so then is viable; `rest =  $\llbracket \perp \rrbracket$` so else is dead
3. In then: `sop(x.n) = sel( $\llbracket C(n: 2) \rrbracket$ , n) =  $\llbracket 2 \rrbracket$` , so $y \mapsto \llbracket 2 \rrbracket$

Shape Propagation

Trace 1: `test() = f(C(2))`

Specialise `f` with $x \mapsto \llbracket C(n: 2) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: 2) \rrbracket$

$y \mapsto \llbracket 2 \rrbracket$

1. let `y`: extend ctx with $y \mapsto \llbracket \perp \rrbracket$
2. if `x` is `C`: `filter( $\llbracket C(n: 2) \rrbracket$ , C) =  $\llbracket C(n: 2) \rrbracket$  so then is viable; rest =  $\llbracket \perp \rrbracket$  so else is dead`
3. In then: `sop(x.n) = sel( $\llbracket C(n: 2) \rrbracket$ , n) =  $\llbracket 2 \rrbracket$ , so  $y \mapsto \llbracket 2 \rrbracket$`
4. `y + 1` folds to $\llbracket 3 \rrbracket$, then Return 3

Cached as `f_C_Lit2`; body folds entirely to the constant 3.

Shape Propagation

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise `f` with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

Shape Propagation

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise `f` with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let `y: y` $\mapsto \llbracket \perp \rrbracket$

Shape Propagation

Trace 2: test2(dyn) = f(C(dyn))

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise f with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket C(n: \text{dyn}) \rrbracket, C) = \llbracket C(n: \text{dyn}) \rrbracket$ so then is viable; $\text{rest} = \llbracket \perp \rrbracket$ so else is dead

Shape Propagation

Trace 2: `test2(dyn) = f(C(dyn))`

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise `f` with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

$y \mapsto \llbracket \text{dyn} \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket C(n: \text{dyn}) \rrbracket, C) =$
 $\llbracket C(n: \text{dyn}) \rrbracket$ so then is viable; $\text{rest} = \llbracket \perp \rrbracket$
so else is dead
3. In then: $\text{sop}(x.n) = \text{sel}(\llbracket C(n: \text{dyn}) \rrbracket, n) =$
 $\llbracket \text{dyn} \rrbracket$

Shape Propagation

Trace 2: test2(dyn) = f(C(dyn))

Argument shape: $\llbracket C(n: \text{dyn}) \rrbracket$.

Specialise f with $x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket C(n: \text{dyn}) \rrbracket$

$y \mapsto \llbracket \text{dyn} \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket C(n: \text{dyn}) \rrbracket, C) = \llbracket C(n: \text{dyn}) \rrbracket$ so then is viable; $\text{rest} = \llbracket \perp \rrbracket$ so else is dead
3. In then: $\text{sop}(x.n) = \text{sel}(\llbracket C(n: \text{dyn}) \rrbracket, n) = \llbracket \text{dyn} \rrbracket$
4. $y + 1$: cannot fold ($\llbracket \text{dyn} \rrbracket + 1$); emit code, result $\llbracket \text{dyn} \rrbracket$

Cached as f_C_Dyn ; body keeps the $y = x.n$ assignment.

Shape Propagation

Trace 3: test3() = f(0)

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise f with $x \mapsto \llbracket 0 \rrbracket$.

ctx

$x \mapsto \llbracket 0 \rrbracket$

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

Shape Propagation

Trace 3: test3() = f(0)

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise f with $x \mapsto \llbracket 0 \rrbracket$.

```
1 fun f(x) = mls  
2   let y  
3   if x is C then  
4     y = x.n  
5   else  
6     y = 0  
7   y + 1
```

ctx

$x \mapsto \llbracket 0 \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let y: $y \mapsto \llbracket \perp \rrbracket$

Shape Propagation

Trace 3: `test3() = f(0)`

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise `f` with $x \mapsto \llbracket 0 \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket 0 \rrbracket$

$y \mapsto \llbracket \perp \rrbracket$

1. let `y`: $y \mapsto \llbracket \perp \rrbracket$
2. if `x` is `C`: $\text{filter}(\llbracket 0 \rrbracket, C) = \llbracket \perp \rrbracket$ so then is dead; $\text{rest}(\llbracket 0 \rrbracket, C) = \llbracket 0 \rrbracket$ so else is viable

Shape Propagation

Trace 3: `test3() = f(0)`

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise `f` with $x \mapsto \llbracket 0 \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket 0 \rrbracket$

$y \mapsto \llbracket 0 \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket 0 \rrbracket, C) = \llbracket \perp \rrbracket$ so then is dead; $\text{rest}(\llbracket 0 \rrbracket, C) = \llbracket 0 \rrbracket$ so else is viable
3. In else: $y \mapsto \llbracket 0 \rrbracket$

Shape Propagation

Trace 3: `test3() = f(0)`

Argument shape: $\llbracket 0 \rrbracket$ (literal).

Specialise `f` with $x \mapsto \llbracket 0 \rrbracket$.

```
1 fun f(x) = mls
2   let y
3   if x is C then
4     y = x.n
5   else
6     y = 0
7   y + 1
```

ctx

$x \mapsto \llbracket 0 \rrbracket$

$y \mapsto \llbracket 0 \rrbracket$

1. let $y: y \mapsto \llbracket \perp \rrbracket$
2. if x is C : $\text{filter}(\llbracket 0 \rrbracket, C) = \llbracket \perp \rrbracket$ so then is dead; $\text{rest}(\llbracket 0 \rrbracket, C) = \llbracket 0 \rrbracket$ so else is viable
3. In else: $y \mapsto \llbracket 0 \rrbracket$
4. $y + 1$ folds to $\llbracket 1 \rrbracket$, then Return 1

Cached as `f_Lit0`.

Shape Propagation

Resulting Cache

After all three calls, the staged module contains:

```
1  module Simple withs
2    fun f_C_Dyn(x) =
3      let y
4        y = x.n
5        y + 1 // dyn case: code is kept
6    fun f_C_Lit2(x) = 3 // fully specialised
7    fun f_Lit0() = 1 // fully specialised
8    fun test() = 3
9    fun test2(dyn) = f_C_Dyn(C$If2(dyn))
10   fun test3() = 1
```

mls

Shape Propagation

Formalization of

Shape Propagation

Dynamic Dispatching

When a method is called on a value with a **union shape**, we split by class and dispatch separately.

```
1 staged class B1(val y) with fun call(x) = x + 2 + y
2 staged class B2(val y) with fun call(x) = x + y
3 staged module M with
4   fun twice(f, x) = f.call(f.call(x))
5   fun pick(x, y, b) = if b then x else y
6   fun f(b) =
7     let m = pick(new B1(2), new B2(3), b)
8     twice(m, 5)
```

Shape Propagation

Trace A: $f(\text{dyn})$

Specialise f with $b \mapsto \llbracket \text{dyn} \rrbracket$.

```
1 fun f(b) = mls  
2   let m = pick(new B1(2),  
3               new B2(3), b)  
3   twice(m, 5)
```

ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

Shape Propagation

Trace A: $f(\text{dyn})$

Specialise f with $b \mapsto \llbracket \text{dyn} \rrbracket$.

```
1 fun f(b) = mls  
2   let m = pick(new B1(2),  
3               new B2(3), b)  
3   twice(m, 5)
```

ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket \perp \rrbracket$

1. let m : extend ctx with $m \mapsto \llbracket \perp \rrbracket$

Shape Propagation

Trace A: $f(\text{dyn})$

Specialise f with $b \mapsto \llbracket \text{dyn} \rrbracket$.

```
1 fun f(b) = mls  
2   let m = pick(new B1(2),  
3     new B2(3), b)  
3   twice(m, 5)
```

ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket \perp \rrbracket$

1. let m : extend ctx with $m \mapsto \llbracket \perp \rrbracket$
2. Call $\text{pick}(\text{new B1}(2), \text{new B2}(3), b)$:
argument shapes are $\llbracket \text{B1}(2) \rrbracket$, $\llbracket \text{B2}(3) \rrbracket$,
 $\llbracket \text{dyn} \rrbracket$. Recurse into pick with a fresh ctx
→ go to Trace B

Shape Propagation

Trace B: `pick(B1(2), B2(3), dyn)`

```
1 fun pick(x, y, b) =  
2   if b then x  
3   else y
```

mls

ctx

fresh ctx for pick

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

Shape Propagation

Trace B: `pick(B1(2), B2(3), dyn)`

```
1 fun pick(x, y, b) = mls
2   if b then x
3   else y
```

ctx

fresh ctx for pick

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

1. if b: b is $\llbracket \text{dyn} \rrbracket$, so both branches viable

Shape Propagation

Trace B: `pick(B1(2), B2(3), dyn)`

```
1 fun pick(x, y, b) = mls  
2   if b then x  
3   else y
```

ctx

fresh ctx for pick

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

1. if b: b is $\llbracket \text{dyn} \rrbracket$, so both branches viable
2. then arm returns x, shape = $\llbracket B1(2) \rrbracket$

Shape Propagation

Trace B: `pick(B1(2), B2(3), dyn)`

```
1 fun pick(x, y, b) =  
2   if b then x  
3   else y
```

mls

ctx

fresh ctx for pick

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

1. if b: b is $\llbracket \text{dyn} \rrbracket$, so both branches viable
2. then arm returns x, shape = $\llbracket B1(2) \rrbracket$
3. else arm returns y, shape = $\llbracket B2(3) \rrbracket$

Shape Propagation

Trace B: `pick(B1(2), B2(3), dyn)`

```
1 fun pick(x, y, b) =  
2   if b then x  
3   else y
```

mls

ctx

fresh ctx for pick

$x \mapsto \llbracket B1(2) \rrbracket$

$y \mapsto \llbracket B2(3) \rrbracket$

$b \mapsto \llbracket \text{dyn} \rrbracket$

1. if b: b is $\llbracket \text{dyn} \rrbracket$, so both branches viable
2. then arm returns x, shape = $\llbracket B1(2) \rrbracket$
3. else arm returns y, shape = $\llbracket B2(3) \rrbracket$
4. Result shape: $\llbracket B1(2) \cup B2(3) \rrbracket$. Cached as pick1. Return to Trace A.

Shape Propagation

Trace A (continued): back in f

```
1 fun f(b) = mls  
2   let m = pick(new B1(2),  
3               new B2(3), b)  
3   twice(m, 5)
```

ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

Shape Propagation

Trace A (continued): back in f

```
1 fun f(b) = mls  
2   let m = pick(new B1(2),  
3               new B2(3), b)  
3   twice(m, 5)
```

ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

1. Bind m to pick's returned shape =
 $\llbracket B1(2) \cup B2(3) \rrbracket$

Shape Propagation

Trace A (continued): back in f

```
1 fun f(b) = mls  
2   let m = pick(new B1(2),  
3               new B2(3), b)  
3   twice(m, 5)
```

ctx

$b \mapsto \llbracket \text{dyn} \rrbracket$

$m \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

1. Bind m to pick's returned shape = $\llbracket B1(2) \cup B2(3) \rrbracket$
2. `twice(m, 5)`: m has **union shape** → go to **Trace C** and specialise as `twice1`

Shape Propagation

Trace C: `twice({B1(2), B2(3)}, 5)`

```
1 fun twice(f, x) =  
2   let tmp  
3   tmp = f.call(x)  
4   f.call(tmp)
```

mls

ctx

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

Recall:

```
1 class B1(y) with  
2   fun call(x) = x + 2 + y  
3 class B2(y) with  
4   fun call(x) = x + y
```

mls

Shape Propagation

Trace C: `twice({B1(2), B2(3)}, 5)`

```
1 fun twice(f, x) = mls  
2   let tmp  
3   tmp = f.call(x)  
4   f.call(tmp)
```

Recall:

```
1 class B1(y) with mls  
2   fun call(x) = x + 2 + y  
3 class B2(y) with  
4   fun call(x) = x + y
```

ctx

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

$tmp \mapsto \llbracket \perp \rrbracket$

1. let tmp: extend ctx with $tmp \mapsto \llbracket \perp \rrbracket$

Shape Propagation

Trace C: `twice({B1(2), B2(3)}, 5)`

```
1 fun twice(f, x) = mls  
2   let tmp  
3   tmp = f.call(x)  
4   f.call(tmp)
```

Recall:

```
1 class B1(y) with mls  
2   fun call(x) = x + 2 + y  
3 class B2(y) with  
4   fun call(x) = x + y
```

ctx

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

$tmp \mapsto \llbracket \{9, 8\} \rrbracket$

1. let tmp: extend ctx with $tmp \mapsto \llbracket \perp \rrbracket$
2. `f.call(x)` on union $\rightarrow$ **split by class**:
 - $f = \llbracket B1(2) \rrbracket$: cache `f.call1()` $\rightarrow \llbracket 9 \rrbracket$
 - $f = \llbracket B2(3) \rrbracket$: cache `f.call1()` $\rightarrow \llbracket 8 \rrbracket$

Shape Propagation

Trace C (continued): second call

```
1 fun twice(f, x) = mls  
2   let tmp  
3     tmp = f.call(x)  
4     f.call(tmp)
```

Recall:

```
1 class B1(y) with fun mls  
  call(x) = x + 2 + y  
2 class B2(y) with fun  
  call(x) = x + y
```

ctx

$f \mapsto \llbracket B1(2) \cup B2(3) \rrbracket$

$x \mapsto \llbracket 5 \rrbracket$

$tmp \mapsto \llbracket \{9, 8\} \rrbracket$

3. $f.call(tmp)$ with tmp in $\llbracket \{9, 8\} \rrbracket$ (now $\llbracket dyn \rrbracket$): split again, body kept as code
- ▶ $f = \llbracket B1 \rrbracket$: cache $f.call2(x) = x + 2 + 2$
 - ▶ $f = \llbracket B2 \rrbracket$: cache $f.call2(x) = x + 3$

Shape Propagation

Resulting Cache

After propagation, M and the staged classes contain:

```
1 module M with mls
2   fun f(b) =
3     let m, tmp1, tmp2
4     tmp1 = new B1(2)
5     tmp2 = new B2(3)
6     m = pick1(tmp1, tmp2, b)
7     twice1(m)
8   fun pick1(x, y, b) =
9     if b then new B1(2) else new B2(3)
```

```
1   fun twice1(f) =
2     let tmp
3     if f is
4       B1 then tmp = f.call1()
5       B2 then tmp = f.call1()
6     if f is
7       B1 then f.call2(tmp)
8       B2 then f.call2(tmp)
```

Shape Propagation

```
1  class B1(y) with mls
2    fun call1() = 9
3    fun call2(x) =
4      let tmp
5        tmp = x + 2
6        tmp + 2
7
8  class B2(y) with
9    fun call1() = 8
10   fun call2(x) = x + 3
```

Printing Staged Block

After all the specialized functions are completed, we need to write the functions to a new file.

```
1 staged module M with
2   val funCache = new Map([
3     ["f1", ...],
4     ["f2", ...],
5     ["g1", ...]
6   ])
```

mls

```
1 module M with
2   fun f1(x, y) = ...
3   fun f2() = ...
4   fun g1() = ...
```

mls

Printing Staged Block

This is a similar process to staging, but done in reverse.

As before, we need extra care when handling symbols.

Motivation	1
Overview	9
Implementation	14
Benchmarking	39
Model-View-Projection transformation	40
Bibliography	41

Model-View-Projection transformation

basically we're pretty good at partially evaluating matrices w .

Time: about 2x (from 2s to 1s, eh...)

Code size: don't worry about it, we'll just do lifting

Bibliography

- [1] W. Taha, “A gentle introduction to multi-stage programming,” in *Domain-Specific Program Generation: International Seminar, Dagstuhl Castle, Germany, March 23-28, 2003. Revised Papers*, 2004, pp. 30–50.
- [2] A. Shali and W. R. Cook, “Hybrid partial evaluation,” in *Proceedings of the 2011 ACM international conference on Object oriented programming systems languages and applications*, 2011, pp. 375–390.